



Capítulo 6

Automatización Web con SeleniumLibrary

Objetivos:

- Describir cómo SeleniumLibrary se integra con Robot Framework para controlar navegadores web de forma automatizada.
- Explicar el papel de WebDriverManager en la gestión automática de controladores de navegador y sus ventajas frente a la configuración manual.
- Interpretar las opciones de sesión y configuración de navegador disponibles en SeleniumLibrary para adaptar la ejecución a distintos entornos.
- La keyword Open Browser es el punto de entrada para cualquier sesión; acepta parámetros como el navegador, el alias de sesión y un objeto de opciones.

6.1: Configuración de Navegador, WebDriverManager, Sesiones y Opciones

- Introducción

SeleniumLibrary conecta Robot Framework con un navegador real (o headless) vía Selenium WebDriver. Desde Selenium 4.6, ya no necesitas instalar manualmente el driver del navegador.

SeleniumLibrary: Puente hacia el Navegador

- ▶ SeleniumLibrary conecta keywords .robot con la API de Selenium WebDriver.
- ▶ Cada navegador requiere un driver binario específico (ChromeDriver, GeckoDriver).
- ▶ Cadena: .robot → SeleniumLibrary → Selenium WebDriver → Driver → Navegador.
- ▶ Más de 100 keywords listos: apertura, interacción, validaciones, capturas.

CADENA DE COMUNICACIÓN: ROBOT FRAMEWORK A NAVEGADOR



Nota: Flujo bidireccional para ejecutar acciones y obtener resultados.

WebDriverManager: Gestión Automática de Drivers

- Sincronizar versión del navegador y del driver era un punto de fricción histórico.
- WebDriverManager descarga y configura el driver correcto automáticamente.
- Selenium 4.6+ incluye selenium-manager nativo sin configuración adicional.
- Elimina roturas de suites por actualizaciones automáticas del navegador.



6.2: Localizadores CSS y XPath: selección robusta, acciones e interacción con formularios

- Introducción

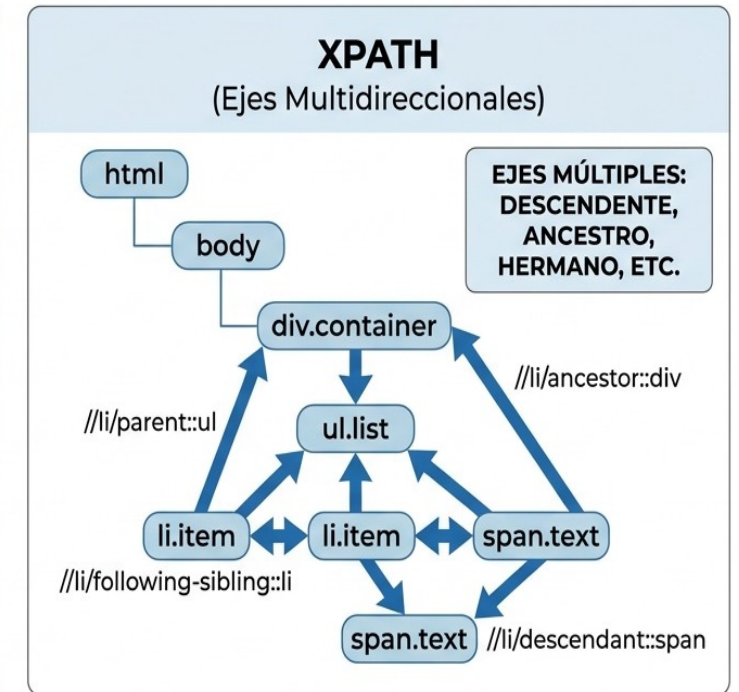
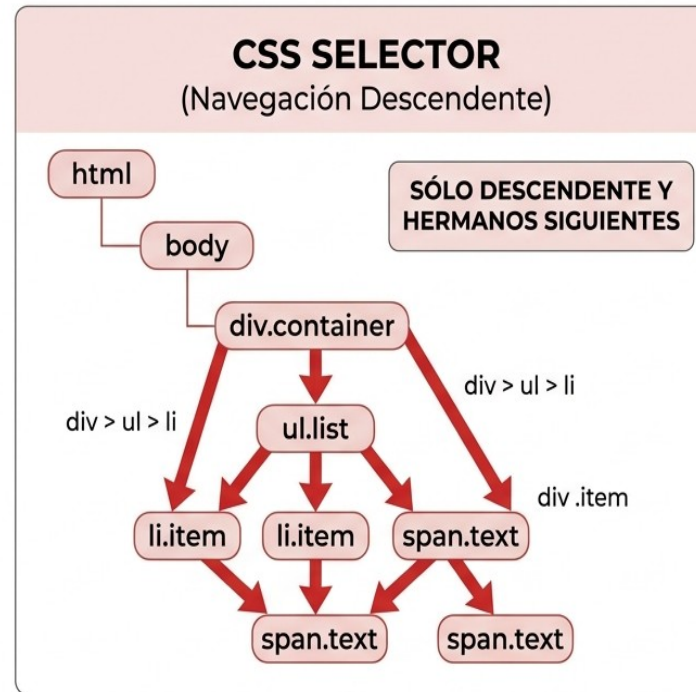
CSS y XPath son los dos lenguajes para localizar elementos.

Cada tipo de control HTML (texto, botón, select) tiene su propia keyword de interacción.

CSS vs XPath: Fundamentos de Localización

- ▶ Un localizador es la dirección para encontrar un elemento en el DOM.
- ▶ CSS Selector: conciso, rápido, ideal para id, class, data-.*.
- ▶ XPath: navega relaciones complejas y busca por texto visible.
- ▶ Regla general: prefiere CSS; usa XPath para relaciones o texto exacto.

COMPARACIÓN DE ESTRATEGIAS DE SELECCIÓN: CSS SELECTOR VS. XPATH EN EL DOM



Prefijos de Localizadores en SeleniumLibrary

Tabla Visual: Comparativa de Localizadores en un Formulario de Login

 HTML Fuente	 Localizador CSS Equivalente	 Localizador XPath Equivalente
<pre><form class="login-form"> <label for="usuario">Usuario</label> <input type="text" id="usuario" name="usuario" placeholder="Introduce tu usuario"> <label for="contraseña">Contraseña</label> <input type="password" id="contraseña" name="contraseña" placeholder="Contraseña segura"> <button type="submit" class="btn-login">Iniciar Sesión</button> </form></pre>	<pre>[id="usuario"] .login-form input[type="password"] .btn-login</pre>	<pre>//input[@id="usuario"] //form/input[@type="password"] //button[contains(text(),'Iniciar Sesión')]</pre>

- ▶ Prefijo css: o xpath: indica la estrategia al motor de Selenium.
- ▶ Sin prefijo, RF resuelve por id o name automáticamente.
- ▶ Prefijos explícitos evitan ambigüedad y mejoran la legibilidad.
- ▶ Ejemplo: css:#login-btn vs xpath://button[text()='Login'].

6.3: Sincronización: Wait Until, Timeouts, Elementos Dinámicos y Estabilidad

- Introducción

Sleep con tiempo fijo es antipatrón. Wait Until espera activamente hasta que se cumple una condición, con timeout — más rápido y más estable.

Sincronización en Automatización Web

- ▶ Los flaky tests ocurren cuando el script interactúa con elementos antes de que estén disponibles.
- ▶ Las apps modernas cargan contenido de forma asíncrona y renderizan elementos progresivamente.
- ▶ Sin sincronización, script y navegador operan de forma independiente causando fallos aleatorios.
- ▶ Las esperas explícitas evalúan condiciones activamente y son más eficientes que pausas fijas.
- ▶ SeleniumLibrary implementa esperas explícitas con sus keywords Wait Until *.
- ▶ Sleep es frágil, desperdicia tiempo y debe usarse solo como último recurso.

Estrategias de Espera: Comparativa

Estrategia	Descripción	Recomendación
Espera implícita	El driver espera globalmente un tiempo fijo ante cualquier búsqueda de elemento	Uso limitado; puede enmascarar problemas reales
Espera explícita	Se espera hasta que una condición específica se cumpla	Recomendada en la mayoría de los casos
Pausa fija (Sleep)	El script se detiene un número de segundos sin verificar condición	Solo como último recurso; frágil y lenta

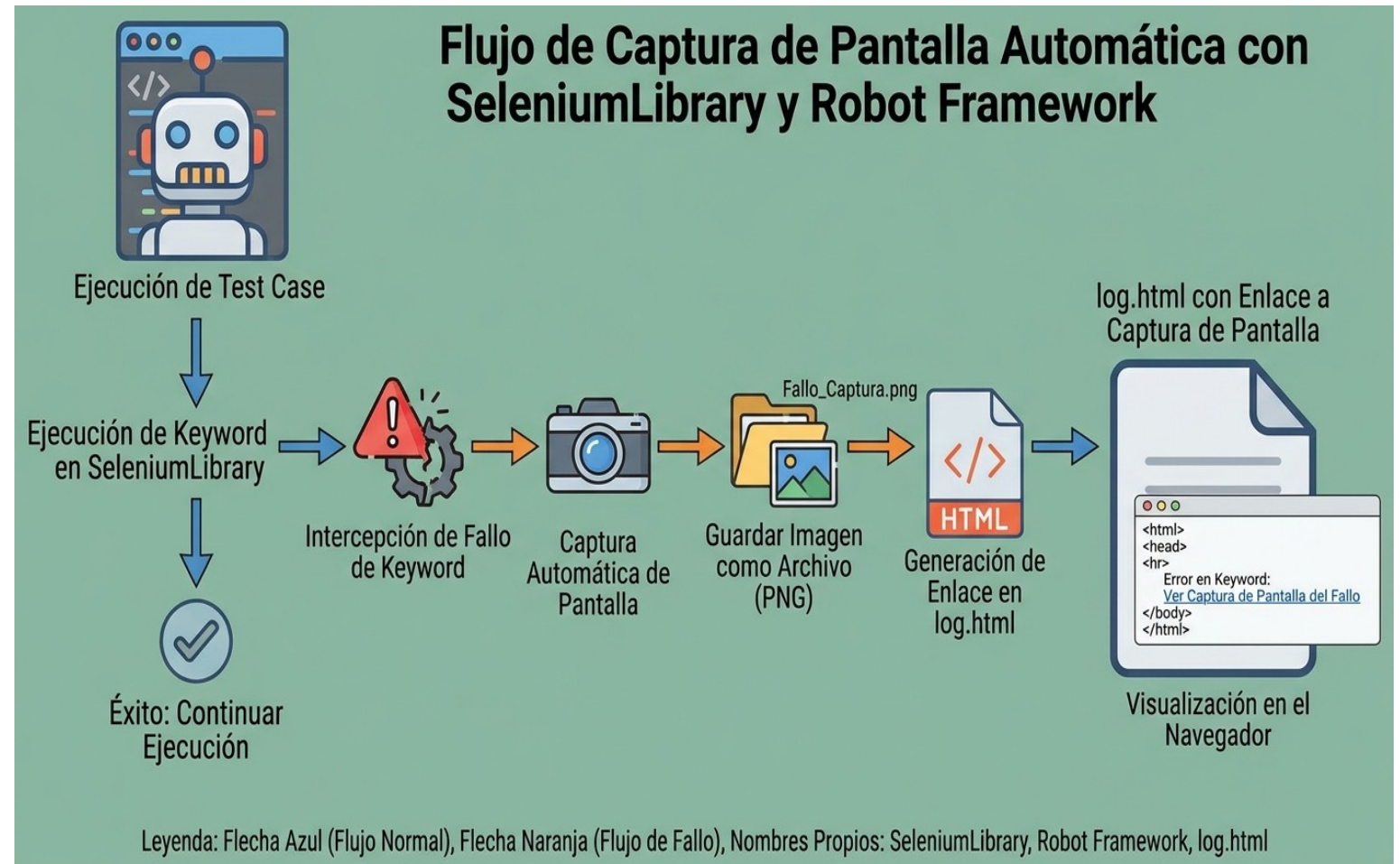
6.4: Evidencias: Screenshots Automáticos, Manejo de Alertas, Frames e Iframes

- Introducción

Capturar pantalla (en fallo o siempre) da evidencia de qué ocurrió. Las alertas de JavaScript no son parte del DOM: requieren Handle Alert, no un clic normal.

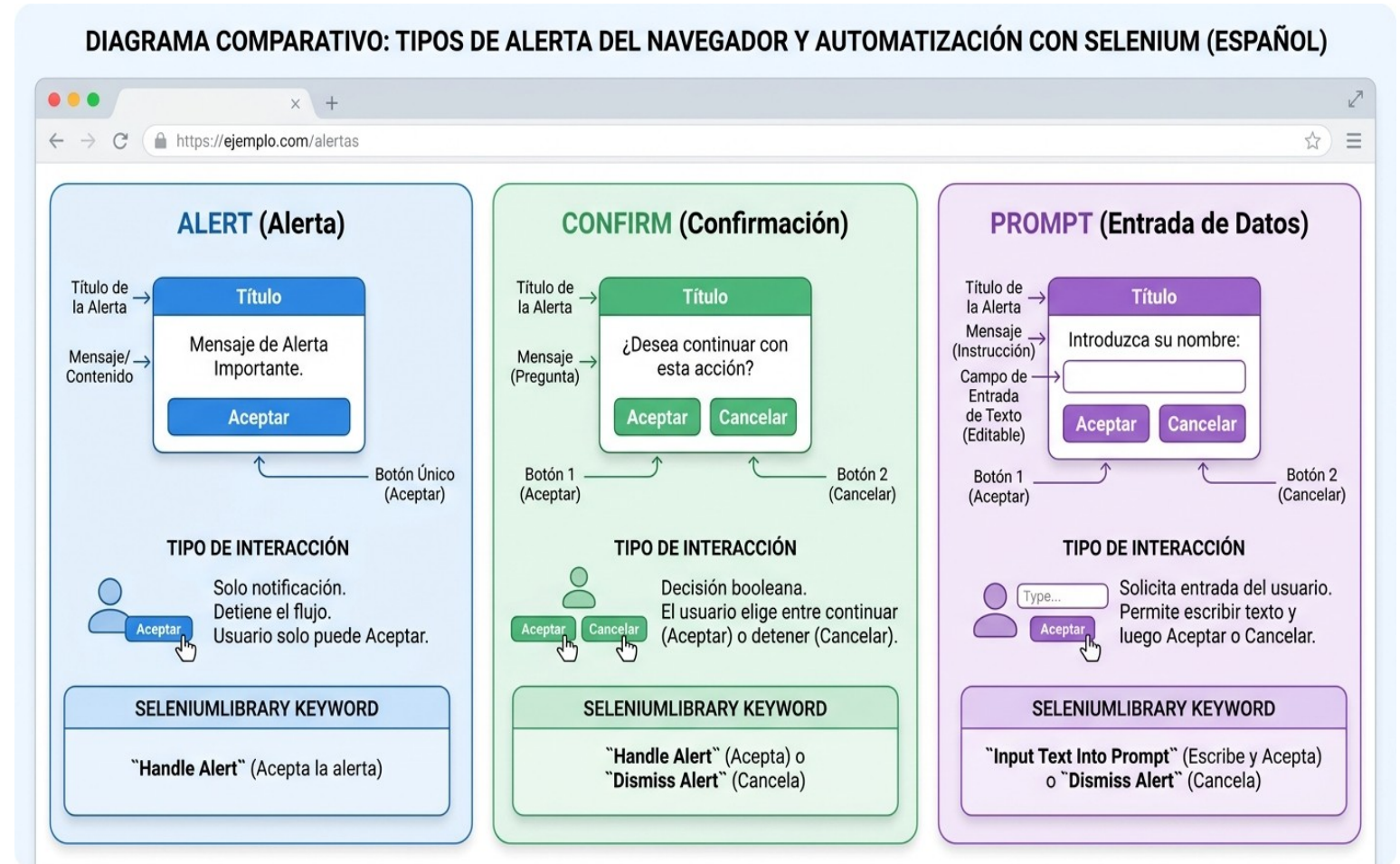
Screenshots en SeleniumLibrary

- ▶ Capturas manuales: Capture Page Screenshot / Capture Element Screenshot.
- ▶ Capturas automáticas: Register Keyword To Run On Failure activa la red de seguridad.
- ▶ Cada fallo genera imagen enlazada en log.html sin configuración extra.
- ▶ Teardown personalizado: captura solo si la prueba falló.



Manejo de Alertas del Navegador

- ▶ alert: mensaje + OK; confirm: OK/Cancelar; prompt: entrada de texto.
- ▶ Alerta no gestionada → UnexpectedAlertPresentException y bloqueo.
- ▶ Handle Alert
action=ACCEPT/DISMISS timeout=5s cubre los tres tipos.
- ▶ Común en portales telecom: confirmar cancelación o portabilidad.



6.5: Patrón Page Object en RF: capa de keywords por página y mantenibilidad

- Introducción

Page Object: cada pantalla tiene su propio archivo Resource con localizadores y keywords. Un cambio de UI se corrige en un solo lugar.

¿Qué es el Patrón Page Object?

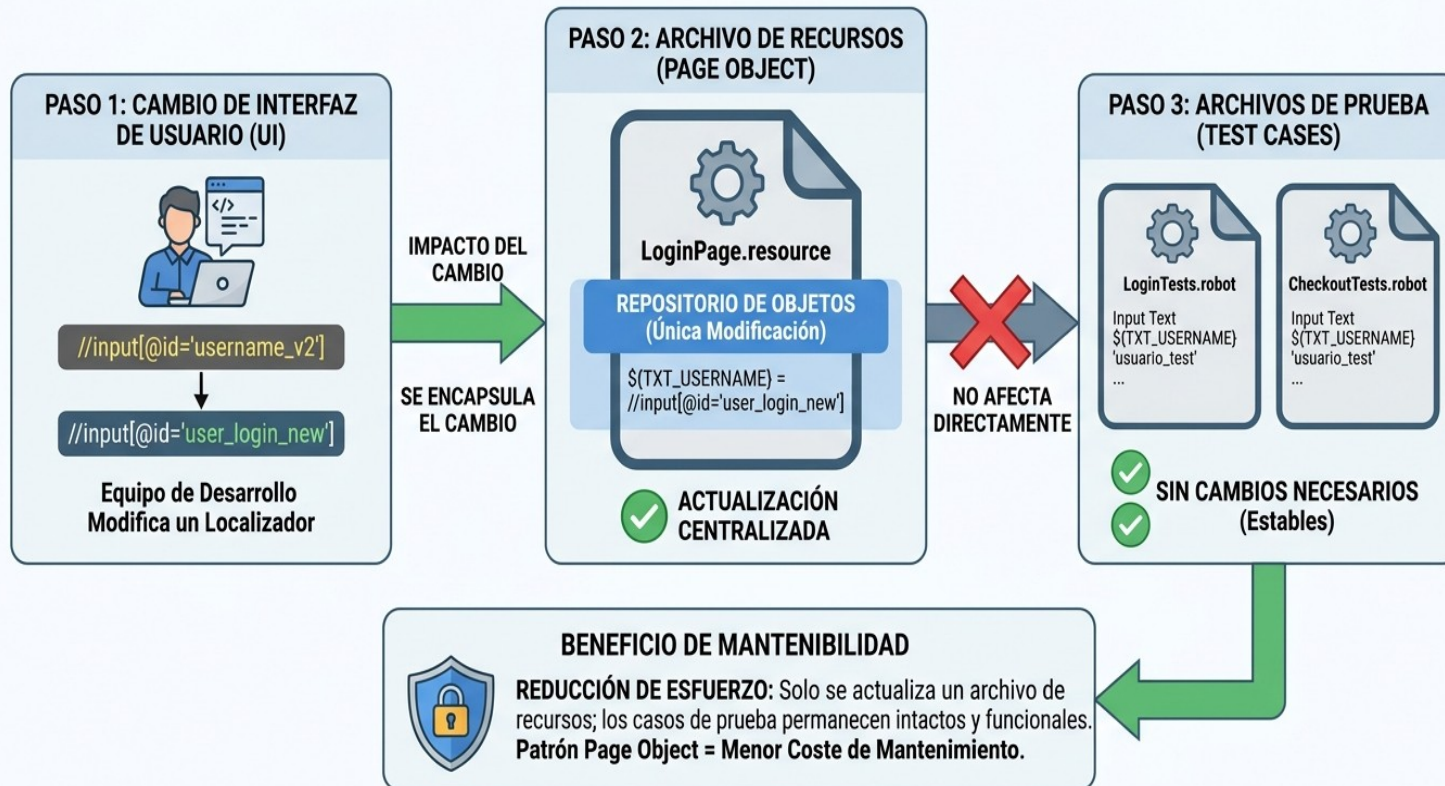
- ▶ Propone que cada pantalla tenga su propia representación en el código de prueba.
- ▶ Encapsula localizadores y acciones de cada página en un único artefacto.
- ▶ Expone solo una interfaz de alto nivel al resto de las suites.
- ▶ Nació en Selenium/Java-Python pero su filosofía es independiente del lenguaje.
- ▶ En Robot Framework se implementa con archivos Resource organizados por página.
- ▶ Evita la duplicación de selectores dispersos en múltiples archivos de prueba.

Dos Capas del Patrón Page Object en RF

Capa	Responsabilidad	Artefacto RF
Capa de página	Localizadores + acciones elementales	Archivo .resource por página
Capa de prueba	Escenarios de negocio y aserciones	Archivo .robot de suite

Mantenibilidad: El Beneficio Principal

DIAGRAMA DE FLUJO: BENEFICIO DE MANTENIBILIDAD DEL PATRÓN PAGE OBJECT



- ▶ Un cambio en la UI se corrige en un solo archivo Resource.
- ▶ Las suites de prueba actúan como documentación de negocio.
- ▶ Nuevos integrantes entienden el proyecto por la estructura.
- ▶ Los selectores técnicos están aislados de los escenarios.

Resumen del capítulo

- **SeleniumLibrary** es la biblioteca estándar de Robot Framework para automatización web; actúa como puente entre los *keywords* .robot y la API de Selenium WebDriver.
- La keyword Open Browser es el punto de entrada para cualquier sesión; acepta parámetros como el navegador, el alias de sesión y un objeto de opciones.
- **WebDriverManager** y el nuevo **Selenium Manager** (Selenium 4.6+) eliminan la necesidad de gestionar manualmente los binarios de driver, resolviendo el problema de compatibilidad de versiones.
- Las **opciones de navegador** (ChromeOptions, FirefoxOptions) permiten configurar argumentos críticos como --headless, tamaño de ventana y parámetros de seguridad necesarios en entornos Docker o CI/CD.
- Es posible mantener **múltiples sesiones activas** simultáneamente usando el parámetro alias y la keyword Switch Browser.



Resumen del capítulo

- Centralizar la configuración del navegador en un archivo resource reutilizable es una buena práctica que facilita el mantenimiento y la adaptación a distintos entornos.
- **CSS Selector** es conciso y rápido; ideal para atributos directos (id, class, data-*). **XPath** es más expresivo y permite navegar relaciones entre elementos y buscar por texto visible.
- Los localizadores más robustos se basan en atributos estables como id, name o data-testid. Evita rutas absolutas y posiciones numéricas que cambian con frecuencia.
- SeleniumLibrary ofrece keywords específicas para cada tipo de control de formulario; usar la keyword correcta mejora la legibilidad y la confiabilidad de la prueba.
- Input Password debe usarse siempre para campos de contraseña para proteger datos sensibles en los reportes.



Práctica 11: Automatización de flujo de login y navegación E2E

Objetivo:

Implementar un flujo completo de autenticación y navegación usando localizadores CSS con waits explícitos y screenshot automático en fallo.

Guía completa: [robfram-guide/Capitulo06/Practica11.md](#)



Práctica 12: Suite web con Page Object y captura de evidencias completas

Objetivo:

Estructurar una suite bajo patrón Page Object, con captura de evidencias en cada test y manejo de alertas de JavaScript.

Guía completa: [robfram-guide/Capitulo06/Practica12.md](#)



Referencias Bibliográficas

Capítulo 6: Automatización Web con SeleniumLibrary

- ▶ <https://robotframework.org/SeleniumLibrary/SeleniumLibrary.html>
- ▶ <https://github.com/robotframework/SeleniumLibrary>
- ▶ <https://www.selenium.dev/documentation/webdriver/browsers/>
- ▶ https://github.com/SergeyPirogov/webdriver_manager
- ▶ <https://www.selenium.dev/blog/2022/introducing-selenium-manager/>